

Personal Web Portfolio

A technical documentation for the design and development process of my personal portfolio website.

Developer

Juan José Castro Zúñiga | Full-Stack DevOps Engineer

Date

May 18, 2026

Main technologies & Frameworks

Azure App Service · Node.js · React · Docker · GitHub



WEB DEVELOPMENT	3
OVERVIEW	3
UI	3
PROJECT STRUCTURE	4
CI/CD CONFIGURATION	6
CONTAINERIZATION ON DOCKER	6
AZURE CLOUDE	7
GITHUB ACTIONS	10

Web development

Overview

Created a project using React framework, first hosting locally the website via VITE service and using GitHub as [repository](#).

UI

For the UI, I selected the following color palette to induce professionalism and part of my personality:

#0a0a0a Background	#141414 Cards	#252525 Borders	#cc3333 Primary accent
#f0ece6 Primary text	#b8a082 Logo	#8a8a8a Secondary text	

The website is distributed within a “One Page Design” and the NavBar is a “Sticky” one as it is always visible. This allows for a quick overview of my

profile as the target audience is focused on recruiters and people seeking professional services.

The website is distributed on seven components (Intro, About me, Career, Education, Skills & Certs, Projects and Contact) each one developed on different JSXs. The NavBar possesses access to each of these segments and allows to return back to the top easily following Nielsen heuristics. The footer has my name and the last year the website was updated.

Project Structure

The project follows a modular, component-based structure, where all core logic is organized inside the /src directory.

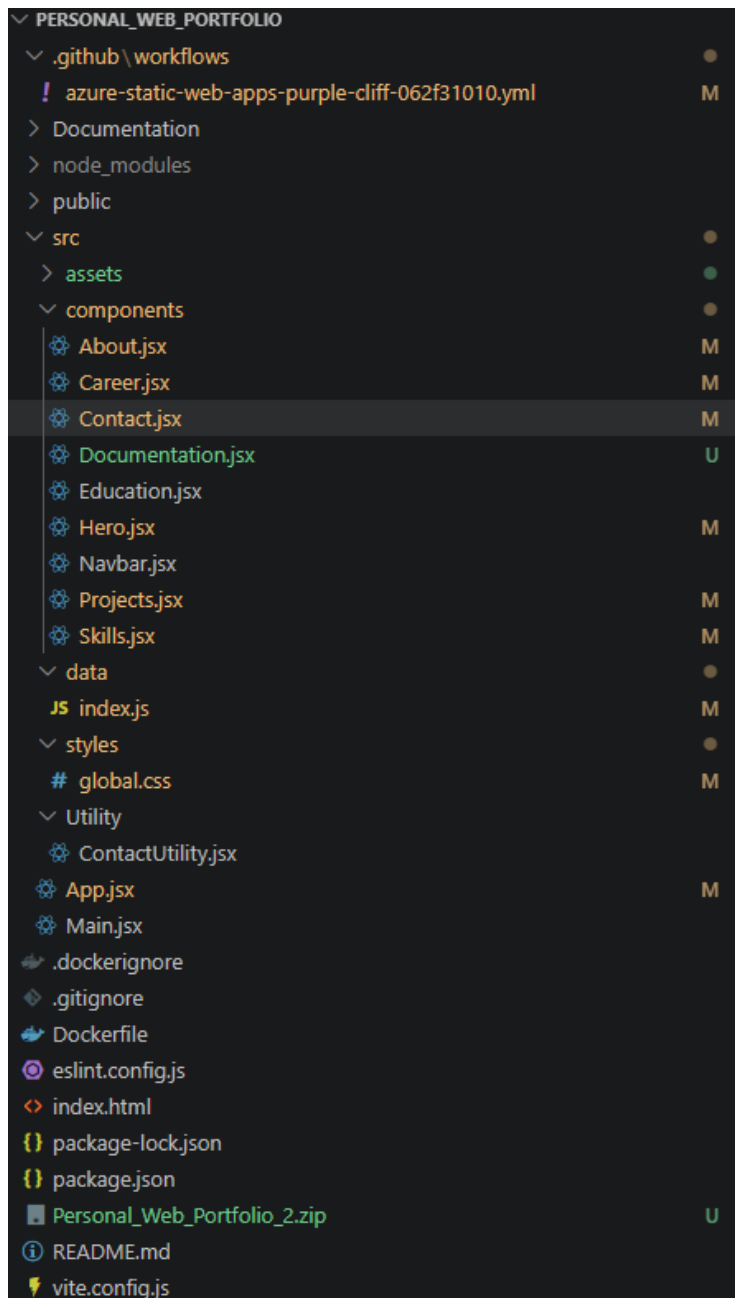
Assets such as images are stored in /assets, while reusable React components are grouped in /components.

Structured data is separated into /data within the Index.j, keeping content independent from UI logic.

Global styles are handled in /styles to ensure visual consistency.

App.jsx defines the main layout, and Main.jsx handles application rendering.

This structure improves readability, maintainability, and scalability.



CI/CD configuration

Containerization on Docker

Created Dockerfile and .dockerignore into React project root. Prepared local environment to test and iterate the containers within a WSL 2 VM.

The container was created and tested locally:

```
C:\Personal_Web_Portfolio>docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED        STATUS        PORTS                               NAMES
db23ed0ad93f   personal_web_portfolio:latest  "/docker-entrypoint..."  8 seconds ago  Up 8 seconds  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  portfolio-prod
```

.docker ignore makes sure unnecessary files for contained versions of the website don't add up space to the container nor cause compatibility issues:

- node_modules
- .git
- .github
- dist
- Build
- .env

Once this was completed, the image was uploaded to the GitHub repository.

Azure Cloude

On the Azure cloud environment, created the Subscription “PersonalProjects” to handle this portfolio and other public projects.

Name	Type	Last Viewed
 PersonalProjects	Subscription	22 hours ago

Created resource group rg-portfolio-prod to encapsulate the resources related to this project. Central US was chosen due to larger availability of free plan resources and geographical location.

```
PS C:\Users\Juan Jose> az group create --name rg-portfolio-prod --location centralus
{
  "id": "/subscriptions/9bc8a60b-2104-4576-8745-343a0fd74df3/resourceGroups/rg-portfolio-prod",
  "location": "centralus",
  "managedBy": null,
  "name": "rg-portfolio-prod",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
```

☐ Name ↑	Subscription	Location
☐  rg-portfolio-prod	... PersonalProjects	Central US

Added an app service plan based on Linux using SKU B1 for a low-cost website able to hold custom domains, we can take advantage of this service tier due to the nature of this project.

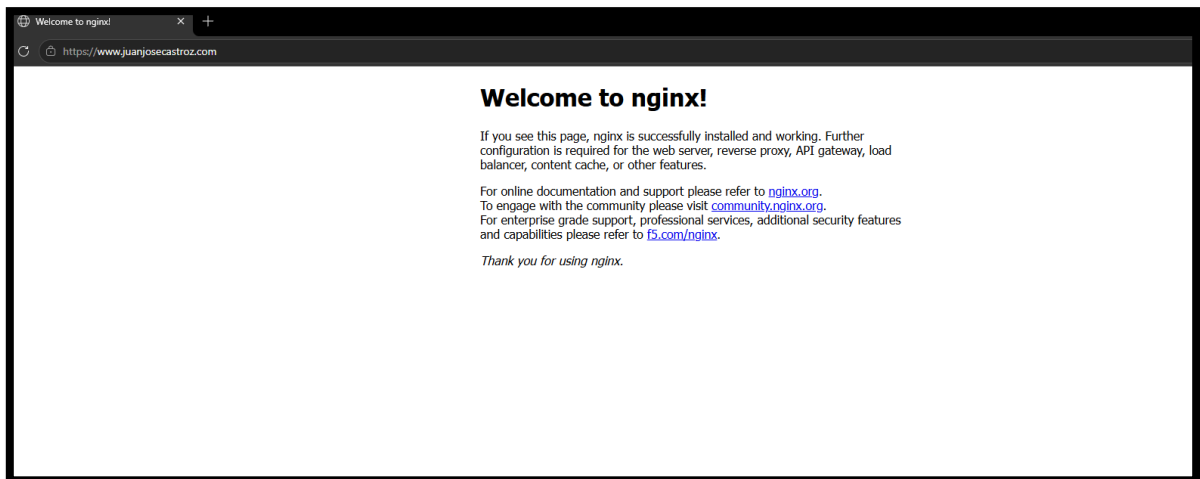
```
PS C:\Users\Juan Jose> az appservice plan create --name plan-portfolio-b1 --resource-group rg-portfolio-prod --sku B1 --is-linux
Creating App Service Plan 'plan-portfolio-b1' (Linux).
{
  "asyncScalingEnabled": false,
  "elasticScaleEnabled": false,
  "freeOfferExpirationTime": "2026-06-18T22:29:27.7533333",
  "geoRegion": "Central US",
}
```

☐	Name ↑	Type	Location
☐	 plan-portfolio-b1	... App Service plan	Central US

Added the webapp within the app service plan previously created. It currently contains an nginx placeholder while we configure our container.

```
PS C:\Users\Juan Jose> az webapp create --name juanjosecastroz --resource-group rg-portfolio-prod --plan plan-portfolio-b1 --deployment-container-image-name nginx:alpine
Option '--deployment-container-image-name' has been deprecated and will be removed in a future release.
{
  "autoGeneratedDomainNameLabelScope": null,
```

Added custom domain linked to GoDaddy purchased domain juanjosecastroz.com. The domain is bound to an SNI SSL Azure certificate created specifically for this custom domain.



Added a static we app connecting my current GitHub repository using the free plan and taking branch “main” as production.

Hosting plan

The hosting plan dictates your bandwidth, custom domain, storage, and other available features. [Compare plans](#)

Plan type

- ☒ Free: For hobby or personal projects
☐ Standard: For general purpose production apps

Deployment details

Source

- ☒ GitHub ☐ Azure DevOps ☐ Other

GitHub account

elyeye4

[Change account](#)



i If you can't find an organization or repository, you might need to enable additional permissions on GitHub. You must have write access to your chosen repository to deploy with GitHub Actions. ×

Organization *

elyeye4

Repository *

Personal_Web_Portfolio

Branch *

main

Build Details

Enter values to create a GitHub Actions workflow file for build and release. You can modify the workflow file later in your GitHub repository.

Build Presets

React (detected)

i These fields will reflect the app type's default project structure. Change the values to suit your app. [Learn more](#)

App location *

/

Api location

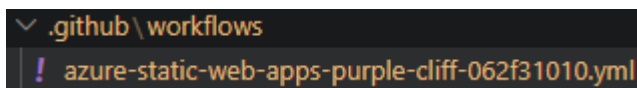
e.g. "api", "functions", etc...

Output location

build

GitHub Actions

The deployment workflow of this portfolio is entirely automated using GitHub Actions. By implementing a custom continuous integration and continuous deployment configuration file, the manual process of building and deploying the application was eliminated.



```
.github\workflows  
! azure-static-web-apps-purple-cliff-062f31010.yml
```

The workflow operates through three core steps:

Trigger: Every time a new commit is pushed to the main branch. GitHub automatically detects the change and triggers the action.

Building process: GitHub provides a clean, temporary Linux virtual machine. It checks out the latest code version, installs the necessary dependencies, and executes Vite's optimized production build engine. This compiles the source React code into a static asset stored in the /dist directory.

The Deployment: Using Microsoft's official Static Web Apps deployment action, the pipeline securely uploads the compiled /dist folder directly to Azure's CDN.

In addition, the workflow configuration includes an explicit "skip_api_build: true" indication. This prevents the deployment engine from scanning for backend components, optimizing build speeds down to just a few seconds.